COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION: CPE21S2**	
DATE PERFORMED:	7/10/2026	**DATE SUBMITTED:**	7/15/26
NAME: ESTORCO, CLINT HOVEN A.		**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 — **Seatwork 1.1 Algorithm, Flowchart, Pseudocode**

1. PROCEDURE OUTPUT

ILO A

INPUT

```cpp
#include <iostream>
#include <iomanip>

// Function prototypes
int sumOfTwoNum(int fNum, int sNum);
void compare(int Num1, int Num2);
int LogicalOperation(bool A, bool B);

int main() {
    // Display the sum of two numbers
    std::cout << "The sum of two numbers is: " << sumOfTwoNum(27, 23) << std::endl;

    //  Compare two numbers
    compare(27, 23);

    // Added a test call to complete the LogicalOperation function
    std::cout << "\n--- Logical Operations Test ---" << std::endl;
    int result = LogicalOperation(true, false);

    return 0;
}

// Function that takes two numbers and returns their sum
int sumOfTwoNum(int fNum, int sNum) {

    return fNum + sNum;
}

// Function to determine if Num1 is greater, less than, or equal to Num2
void compare(int Num1, int Num2) {
    if (Num1 > Num2) {
        std::cout << "Value A is greater than Value B" << std::endl;
    }

    else if (Num1 < Num2) {
        std::cout << "Value A is less than Value B" << std::endl;
    }
    else {
        std::cout << "Value A is equal to Value B" << std::endl;
    }
}


int LogicalOperation(bool A, bool B) {
    // boolalpha forces the console to print "true/false" instead of "1/0"
    std::cout << std::boolalpha;

    std::cout << "A AND B (A && B) is: " << (A && B) << std::endl;
    std::cout << "A OR B (A || B)  is: " << (A || B) << std::endl;
    std::cout << "NOT A (!A)       is: " << (!A) << std::endl;


    // Returns 1 (true) if the overall combination is valid/true, otherwise 0
    return (A || B);
}
```

OUTPUT

```
C:\Users\analy\Downloads\IL(
The sum of two numbers is: 50
The A is greater than B

--------------------------------
Process exited after 0.02928 seconds with return value 0
Press any key to continue . . .
```

COMPUTER ENGINEERING

QUEZON CITY

CODE ANALYSIS:

In this code the control structures i have is the if else block to evaluate relation operations between two numerical constant (The value A and the VAlue B), std::bootalpha configure the standard output stream, forcing boolean true or false results to print as text instead of a binary ones and zeros.

ILO B

INPUT

```
#include <iostream>
#include <iomanip>

class Triangle {
    private:
     double totalAngle, angleA, angleB, angleC;

    public:
     Triangle(double A, double B, double C);
     void setAngle(double A, double B, double C);
     const bool validateTriangle();

};
int main() {

    Triangle set1(40,50,180);
    if(set1.validateTriangle()){
        std::cout <<"The angle is a valid triangle \n.";
    }else{
        std::cout<<"The shape is NOT a triangle \n.";
    }


    return 0;
}
```

```
Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
```

OUTPUT

```
C:\Users\analy\Downloads\IL(
The shape is NOT a triangle
.
--------------------------------
Process exited after 0.06979 seconds with return value 0
Press any key to continue . . .
```

CODE ANALYSIS:

The validateTriangle is the one that validates whether the internal sum of the angle satisfies a geometric limit. In the output test case initializing the shape with angles summing to 270 degrees correctly triggers the validation logic to report that the shape is not a valid triangle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C no.1

INPUT

```cpp
#include <iostream>

// It can now swap doubles, floats, or chars, not just integers.
template <typename T>
void swapNumbers(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}
int main() {
    int num1, num2;

    // Explicitly using the std:: prefix for all standard library functions
    std::cout << "Enter first number: ";
    std::cin >> num1;

    std::cout << "Enter second number: ";
    std::cin >> num2;

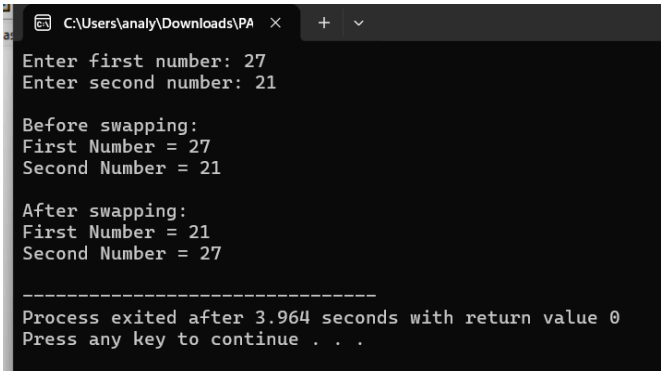
    std::cout << "\nBefore swapping:\n";
    std::cout << "First Number = " << num1 << std::endl;
    std::cout << "Second Number = " << num2 << std::endl;

    swapNumbers(num1, num2);

    std::cout << "\nAfter swapping:\n";
    std::cout << "First Number = " << num1 << std::endl;
    std::cout << "Second Number = " << num2 << std::endl;

    return 0;
}
```

OUTPUT



CODE ANALYSIS:
This program showcases generic programming using a template function to swap the values of two variables of the same data type. This code eliminates redundant code by allowing the same swap logic work seamlessly with integers, floats, or character types.

ILO C no.2

INPUT	OUTPUT

```cpp
#include <iostream>

// Function to convert Kelvin to Fahrenheit
float kelvinToFahrenheit(float kelvin) {

    return (kelvin - 273.15) * (9.0 / 5.0) + 32.0;
}

int main() {
    float kelvin;

    std::cout << "Enter Temperature in Kelvin: ";
    std::cin >> kelvin;

    // Added input validation for absolute zero
    if (kelvin < 0) {
        std::cout << "Error: Temperature cannot be below Absolute Zero (0 Kelvin)." << endl;
        return 1; // Exit with an error code
    }

    float fahrenheit = kelvinToFahrenheit(kelvin);
    std::cout << kelvin << " Kelvin is equal to " << fahrenheit << " Fahrenheit." << Std::endl;

    return 0;
}
```

```
C:\Users\analy\Downloads\PA
Enter Temperature in Kelvin: 190
190 Kelvin is equal to -117.67 Fahrenheit.

--------------------------------
Process exited after 4.41 seconds with return value 0
Press any key to continue . . .
```

CODE ANALYSIS:
This activity implements a mathematical function to convert temperature readings from kelvin to Farenheit, it uses the standard conversion formula to dynamically calculate and return the equivalent fahrenheit value.

ILO C no.3

INPUT

```cpp
#include <iostream>
#include <cmath>
#include <iomanip> // Added for output formatting


// Function to calculate the distance between two points
double calculateDistance(double x1, double y1, double x2, double y2) {

    double dx = x2 - x1;
    double dy = y2 - y1;
    return sqrt(dx * dx + dy * dy);

}

int main() {
    double x1, y1, x2, y2;

    std::cout << "Enter coordinates of first point (x1 y1): ";
    std::cin >> x1 >> y1;

    std::cout << "Enter coordinates of second point (x2 y2): ";
    std::cin >> x2 >> y2;

    double distance = calculateDistance(x1, y1, x2, y2);

    // Formatted the output to exactly 2 decimal places
    std::cout << fixed << setprecision(2);
    cstd::out << "The distance between the two points is: " << distance << std::endl;

    return 0;
}
```

OUTPUT

```
C:\Users\analy\Downloads\PA
Enter coordinates of first point (x1 y1): 27
23
Enter coordinates of second point (x2 y2): 22
12
The distance between the two points is: 12.08

--------------------------------
Process exited after 9.861 seconds with return value 0
Press any key to continue . . .
```

CODE ANALYSIS:

This program utilizes the standard math library to calculate the straight line euclidean distance between two points in a cartesian coordinates system.

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

ILO C no.4

INPUT

```cpp
#include <iostream>
#include <cmath>

class Triangle {
    private:
        // uppercase for angles, lowercase for sides
        double A, B, C;
        double a, b, c;

    public:

        Triangle(double angleA, double angleB, double angleC, double sideA, double sideB, double sideC) {
            setValues(angleA, angleB, angleC, sideA, sideB, sideC);
        }

        // Sets or changes the triangle's values
        void setValues(double angleA, double angleB, double angleC, double sideA, double sideB, double sideC) {
            A = angleA; B = angleB; C = angleC;
            a = sideA;  b = sideB;  c = sideC;
        }

        // Validates that the angles make sense
        bool validateTriangle() {
            return (A + B + C == 180 && A > 0 && B > 0 && C > 0 && a > 0 && b > 0 && c > 0);
        }

        // Calculates perimeter
        double getPerimeter() {
            return a + b + c;
        }


        double getArea() {
            double s = getPerimeter() / 2.0; // semi-perimeter
            return std::sqrt(s * (s - a) * (s - b) * (s - c));
        }

        // Prints the classification of the triangle
        void printType() {
            if (A < 90 && B < 90 && C < 90) {
                std::cout << "The triangle is an acute triangle.\n";
            } else if (A == 90 || B == 90 || C == 90) {
                std::cout << "The triangle is a right triangle.\n";
            } else {
                std::cout << "The triangle is an obtuse triangle.\n";
            }
        }
};

int main() {
    // Creating a triangle object
    Triangle set1(40, 30, 110, 8, 7, 5);

    if (set1.validateTriangle()) {
        std::cout << "The shape is a valid triangle.\n";
        std::cout << "Perimeter of the triangle: " << set1.getPerimeter() << "\n";
        std::cout << "Area of the triangle: " << set1.getArea() << "\n";
        set1.printType();
    } else {
        std::cout << "The shape is NOT a valid triangle.\n";
    }

    return 0;
}
```

OUTPUT

```
C:\Users\analy\Downloads\IL(
The shape is a valid triangle.
Perimeter of the triangle: 20
Area of the triangle: 17.3205
The triangle is an obtuse triangle.

--------------------------------
Process exited after 0.04156 seconds with return value 0
Press any key to continue . . .
```

CODE ANALYSIS:

This program extends the initial triangle class by tracking both its three internal angles and its three external side lengths. It uses the Heron's formula within a member function to calculate the exact area of the triangle. Another member of function assesses the internal angles to classify the triangle as acute, right angled or obtuse angle.

COMPUTER ENGINEERING

QUEZON CITY

CONCLUSION:

To conclude everything, the sequence of laboratory sessions covering ILO A to ILO C clearly progresses from procedural to advanced design principles in object-oriented C++. They provide a solid foundation in data encapsulation. The supplemental tasks in ILO C use these concepts to work with generic templates for swapping data with minimal memory usage, calculate coordinates of two points in the plane, and calculate engineering formulas, such as Heron's formula. Finally, this summary of validation features generalizes to a critical emphasis on reliability and effective edge valve management in the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini and grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and code assistance, and code analyzation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI is used for code assistance on how to put decimals on a code, having an error, and analysis only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.